CS201 Spring 2025 — Lecture Notes

Clarissa Littler

Spring 2025

Preface

Spring 2025 was an asynchronous, video-based version of CS201, so the shape of these notes is a little different from a normal term. Instead of one entry per in-person meeting, this document follows the way the class repo is actually organized: a few "video" lectures up front that walk you through the core ideas, then deeper topical directories (`assembly/`, `forking/`, `threading/`) that correspond to the longer-form material I recorded as we went. The final section pulls together the bits and bobs from the one synchronous in-class meeting we had on April 10. Each section pairs the agenda I worked off of with the code we actually wrote, so if you watched the video and want to re-read the bullet points, or you read the bullet points and want to see the code, the whole arc should be in one place. The source files live in the topic-named subdirectories of the class repo, and every snippet quoted below is verbatim from one of those files.

Contents

1 Video 1 — C, Bits, and Two's Complement

This first video was the orientation: who I am, how the class works, and a sprint through the parts of C that tend to surprise people coming from C++. By the end of the video we'd warmed up with a few small C programs and built up enough machinery to talk about how integers are actually stored.

What is this class?

It's my own personal admixture of four things:

- C
- x86-64 assembly
- Concurrency
- Operating systems theory and practice

Because we're running this term asynchronously, the rhythm is: watch the video, read the corresponding agenda in the repo, then sit down with the source files and actually compile them. The repo is the textbook.

C: a brief summary thereof

If you're coming from C++, the big shocks are mostly subtractive:

- No pass-by-reference. You get pass-by-value or you get pointers, full stop.
- No `bool` type — nonzero is truthy, zero is falsy, and there's no native `true`/`false`.
- No `string` type. You get `char*` (or `char[]`).
- `struct` always has to say "`struct`" in the type — there's no implicit naming the way there is in C++.
- I/O is `printf` and `scanf`, not streams.
- Compile with `gcc`, not `g++`.
- Memory management is `malloc`/`free`, not `new`/`delete`.

The first three programs were just to get something on the screen. `hello.c` is the obligatory hello-world:

```c
#include <stdio.h>

int main(){
  printf("Hello world!\n");
  return 0;
}
```

`echo.c` adds a `scanf` so we can see how input works:

```c
int main(){
  char msg[256];
  printf("Enter a message to echo (no longer than 256 characters): ");
  scanf("%s",msg);
  printf("You said: %s\n",msg);
  return 0;
}
```

And `echoint.c` is the same idea but with an integer, which makes the `&` sting a bit:

```c
int main(){
  int num;
  printf("Enter a number to echo: ");
  scanf("%d",&num);
  printf("You said: %d\n",num);
  return 0;
}
```

The ampersand on `&num` is doing real work — `scanf` needs the *address* of the variable so it can write the parsed integer into it. There's no pass-by-reference; you pass a pointer or nothing happens.

Bit munging

The core bitwise operators with their truth-table behavior:

- `&` (AND), `|` (OR), `^` (XOR).
- `>>` right shift (pads with zeroes on the left, the bits that fall off the right are gone).
- `<<` left shift (mirror image: pads with zeros on the right, drops bits off the left).

A right shift by m is integer division by 2^m; a left shift by m is multiplication by 2^m. (Worth remembering — it's the easiest "optimization" you can read in compiled code.)

`bitMunger.c` is where these ideas come together. It prints all 32 bits of an `int`, asks you which bit to flip, flips it, and loops:

```c
void printBits(int n){
  for(int i = 31; i >= 0; i--){
    printf("%d",(n >> i) & 1);
  }
  printf("\n");
}

void flipBit(int* n, int c){
  *n = (*n) ^ (1 << c);
}
```

`printBits` walks from bit 31 down to bit 0; the `(n >> i) & 1` idiom shifts the bit you want into the lowest position and then masks off everything else. `flipBit` is the canonical XOR trick: XOR-ing a bit with 1 flips it, XOR-ing with 0 leaves it alone, so XOR-ing against `(1 << c)` flips exactly the c-th bit.

`sizeof` and the shape of types

`sizeof1.c` prints the size of every primitive plus a struct and a pointer:

```c
struct Thingy {
  int thing1;
  double thing2;
};

int main(){
  printf("The size of a char is: %ld\n",sizeof(char));
```

```
  printf("The size of an int is: %ld\n",sizeof(int));
  printf("The size of a float is: %ld\n",sizeof(float));
  printf("The size of a double is: %ld\n",sizeof(double));
  printf("The size of a long is: %ld\n",sizeof(long));
  printf("The size of a thing is: %ld\n",sizeof(struct Thingy));
  printf("The size of a pointer is: %ld\n",sizeof(int*));
  return 0;
}
```

On our 64-bit Linux boxes: char is 1, int is 4, float is 4, long is 8, double is 8, the pointer is 8, and the struct is 16. That last one is the surprise: `int` (4) plus `double` (8) is only 12, but the struct gets padded out to 16 so the `double` sits on an 8-byte boundary. Welcome to alignment.

`malloc` and `free`

We previewed dynamic allocation with `malloc1.c`:

```
int* bigArray = (int*) malloc(100000 * sizeof(int));

for(int i = 0; i < 100000; i++){
  bigArray[i] = i*i;
}

for(int i = 0; i < 100000; i++){
  printf("bigArray[%d] = %d\n",i,bigArray[i]);
}

printf("The address of bigArray is: %p\n",(void*)bigArray);

free(bigArray);
bigArray = NULL;
printf("Now the address of bigArray is: %p\n",(void*)bigArray);
```

Two things to internalize. First, `malloc` doesn't know anything about your type — you tell it how many bytes you want and cast the resulting pointer. Second, after you `free` a pointer, setting it to `NULL` is a hygiene move: you can't accidentally double-free `NULL`, but you can absolutely double-free a stale pointer.

Integer representations

For most bits in an unsigned integer, the n-th bit contributes $b_n \cdot 2^n$ to the total. For *signed* integers we use two's complement, where the top bit's weight flips sign:

$$-b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i.$$

Worked example with an 8-bit two's complement number:

$$10010101_2 = -2^7 + 2^4 + 2^2 + 2^0 = -128 + 16 + 4 + 1 = -107.$$

The range of an n-bit two's complement signed integer is $[-2^{n-1},\ 2^{n-1}-1]$. The range of an n-bit unsigned integer is $[0,\ 2^n-1]$. And as a clean mental check: the all-ones pattern is always -1 in two's complement, because $-2^{n-1}+(2^{n-1}-1)=-1$.

Next time: floating-point.

2 Video 2 — Endianness, Pointer Arithmetic, and IEEE-754 Floats

This video is the floating-point video, with a detour through how multi-byte values actually sit in memory.

Endianness

When we store a multi-byte value (say, a 4-byte `int`) at some address, which byte goes where? That choice is what "endianness" names; it has nothing to do with the order of bits inside a byte, only with the order of bytes in memory.

- *Little-endian*: the least significant byte sits at the lowest address, the most significant at the highest. x86-64 is little-endian.
- *Big-endian*: the most significant byte sits at the lowest address. (Network byte order, lots of older RISC machines.)

Worked example with the 4-byte `int` `0x01020304` stored at base address `0x05`:

- Little-endian: `0x04` at `0x05`, `0x03` at `0x06`, `0x02` at `0x07`, `0x01` at `0x08`.
- Big-endian: `0x01` at `0x05`, `0x02` at `0x06`, `0x03` at `0x07`, `0x04` at `0x08`.

To actually *see* this on our machines, `byteorder.c` casts an `int*` to an `unsigned char*` and walks one byte at a time:

```
int n = 0x89ABCDEF;
unsigned char* c = (unsigned char*) &n;

for(size_t i = 0; i < sizeof(n); i++){
  printf("The %zu th byte of n is: %x \n",i,*(c+i));
}
```

On x86-64 you'll see the bytes printed in the opposite order from how we wrote the literal — classic little-endian. The trick is the `(unsigned char*)` cast: once we have a byte-pointer, pointer arithmetic moves us one byte at a time.

Pointer arithmetic, more carefully

Pointer arithmetic scales with the size of the *pointee*. An `int*` advances 4 bytes per `+1`, a `double*` advances 8, a `char*` advances 1. `pointerarith.c` makes this concrete:

```
int arr1[10];
double arr2[10];
char arr3[10];

printf("The addr of arr1 is %p and the addr of arr1+1 is %p\n",
       (void*)arr1,(void*)(arr1+1));
```

```
printf("The addr of arr2 is %p and the addr of arr2+1 is %p\n",
       (void*)arr2,(void*)(arr2+1));
printf("The addr of arr3 is %p and the addr of arr3+1 is %p\n",
       (void*)arr3,(void*)(arr3+1));
```

You'll see `arr1+1` is 4 bytes past `arr1`, `arr2+1` is 8 past `arr2`, and `arr3+1` is just 1 past `arr3`. This is *the* reason `a[i]` is the same as `*(a + i)`: the compiler scales the index by the element size for you.

Floats: bit-flipping, again

`floatMunger.c` is structurally the same as `bitMunger.c` from video 1, but with two tweaks: we accept a `float` rather than an `int`, and we cast its address to an `int*` so we can poke at the underlying bits:

```
void printBits(int* n){
  for(int i = 31; i >= 0; i--){
    printf("%d",(*n >> i) & 1);
    if(i == 31 || i == 23){
      printf(" ");
    }
  }
  printf("\n");
}

void flipBit(int* n, int c){
  *n = (*n) ^ (1 << c);
}
```

The two stray spaces in `printBits` (after bit 31 and after bit 23) split the printout into the three IEEE-754 fields: sign, exponent, fraction.

IEEE-754, the standard

For 32-bit `float` (the 64-bit version is just "same thing with bigger fields"):

- 1 bit sign — the most significant bit, 0 = positive, 1 = negative.
- 8 bits exponent — interpreted as an unsigned integer, then biased by 127.
- 23 bits fraction (a.k.a. mantissa or significand).

In the normal case the value is

$$(-1)^s \cdot 2^{e-127} \cdot (1 + f),$$

where f is the fraction read as a "rational binary" number: the bit at position k after the implicit binary point contributes 2^{-k}.

Aside: rational binary numbers

The bits to the right of the binary point follow the same pattern as the bits to the left, just with negative powers of 2:

$$b_{-1} \cdot 2^{-1} + b_{-2} \cdot 2^{-2} + b_{-3} \cdot 2^{-3} + \cdots$$

So `0.1111` in binary is $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} = \frac{15}{16}$, and `0.11111` is $\frac{31}{32}$.

Subnormals: when the exponent is all zeros

When all eight exponent bits are zero, the formula changes: the exponent sticks at 2^{-126} and the leading implicit 1 disappears. The value becomes

$$(-1)^s \cdot 2^{-126} \cdot f.$$

This lets us slide *smoothly* down to zero rather than having a big gap between zero and the smallest normal.

A few small observations

We almost never represent reals exactly in floating point. The reason is uncomfortable: the reals are uncountable, and even just the reals in $[0, 1]$ are uncountable — there are strictly more of them than there are programs you could ever write in any language. So no matter how clever the encoding, we're stuck rounding most numbers to the nearest one we can actually express.

`badFloat.c` is the small program that makes this hurt:

```c
int main(){
  float f = 0;

  for(int i = 0; i < 100; i++){
    f += 0.01;
  }

  printf("Our number is %.40f\n",f);
  return 0;
}
```

We'd like this to print 1. It doesn't. `0.01` isn't representable exactly in binary, so each iteration accumulates a tiny rounding error, and 100 iterations later the result has visibly drifted. This is the lie floats tell, and it's why anything that genuinely needs decimal precision (money!) should use a fixed-point or decimal type instead.

For a deeper dive into an alternative encoding, see `positTutorial.org` in the repo.

3 Video 3 — First Assembly: Syscalls, Globals, Comparisons, Loops

This is where we leave C-land. The plan: explain what assembly even is, exit a program by hand via the `exit` syscall, then walk up through arithmetic, global variables, comparisons, and our first loop in assembly.

What happens when you compile, really?

A compiler turns your high-level code into a sequence of bytes that the CPU's hardware decodes as instructions. The set of instructions a CPU understands is its *instruction set architecture* (ISA). Ours is x86-64.

Assembly is a textual encoding of those instructions. It's not really *one* language — not even per ISA — because each *assembler* (the program that turns assembly text into bytes) has its own dialect. On Linux you've got at least two:

- `as` (the GNU assembler), which uses AT&T syntax. A little harder to read at first, but it's the syntax `gdb` prints by default, so you eventually need it.
- `nasm` (the netwide assembler), which uses Intel syntax. Often more pleasant to write.

We'll use `as` all term.

To peek at how a C program compiles, two utilities are handy: `xxd` dumps any file as hex bytes, and `gcc -S file.c` stops gcc at the assembly stage and writes a `.s` file. Running `gcc -S` on `hello.c` produces a `hello.s` that's noisy with `.cfi_*` directives and PLT calls, but the actual work is recognizable: load the address of the string, call `puts@PLT`, return zero.

Registers

Registers are the CPU's smallest, fastest storage. They live on the core itself, so reads and writes happen in a single clock cycle. On x86-64 there are 16 general-purpose registers (plus floating-point and vector registers and the special `%rip`):

- `%rax` — accumulator (return values, implicit operand for `mul`/`div`).
- `%rbx` — base register (historically an array base).
- `%rcx` — counter (used in loops, `rep`).
- `%rdx` — data (I/O, upper half of mul/div).
- `%rsi`, `%rdi` — source/destination index for string operations.
- `%rbp` — base pointer (frame base).
- `%rsp` — stack pointer (top of the stack).
- `%r8`–`%r15` — additional general-purpose registers.
- `%rip` — instruction pointer; you don't write to it directly, but you'll see it in RIP-relative addressing.

Each register holds 8 bytes, so all 16 of them is just 128 bytes of fast storage per core. Memory is roughly four to five orders of magnitude slower than register access; if your code is bouncing values in and out of memory unnecessarily, you can feel it.

Why x86-64?

Because it's what most of the laptops and desktops you'll work on run *at the moment*. ARM is increasingly common (phones, single-board computers, modern Macs), and RISC-V is open-source and growing fast, but the principles transfer between ISAs — once you know one, the others mostly come down to syntax and edge cases.

Our very first program (and why it crashes)

`firstas.s`:

```
        .text
        .global _start

_start:
```

```
       mov $60,%rax # put 60 IN %rax
       mov $-1,%rdi # return -1
       syscall # with 60 in %rax, this syscall will be to exit the program
```

Some of the syntax is non-obvious. `.text` marks the section where code lives. `.global _start` tells the linker that `_start` should be visible from outside this file, because `ld` looks for that symbol as the program entry point (in raw assembly there's no C runtime to call `main`).

The Linux x86-64 syscall convention says: put the syscall number in `%rax`, put arguments in `%rdi`, `%rsi`, `%rdx`, etc., then issue `syscall`. `exit` is syscall 60, and its single argument is the exit status. So `exit(-1)` comes out of the shell as `255`: exit codes are an unsigned byte (0–255), and `-1` as an unsigned byte is `0xff = 255`. Run `echo $?` after running the program to see this.

Arithmetic

`add1.s` doubles a number using `add`:

```
_start:
       mov $11,%rdi # puts the number 11 in %rdi
       add %rdi,%rdi # adds %rdi to itself
       mov $60,%rax
       syscall
```

The crucial syntax note that will trip you up all term: in AT&T syntax, `add S, D` means `D = D + S`. The destination is on the *right*. So `add %rdi,%rdi` is `%rdi += %rdi`, leaving 22. Exit code: 22.

`sub1.s` is the same shape with subtraction:

```
_start:
       mov $11,%rdi
       sub $20,%rdi # %rdi = %rdi - 20
       mov $60,%rax
       syscall
```

That ends with -9 in `%rdi`, which the shell will report as $256 - 9 = 247$.

Global variables: three flavors

We built up to RIP-relative addressing in three steps.

`data0.s` declares a global with `.long` and uses an absolute address:

```
       .section .data
num1: .long 200

       .section .text
       .global _start

_start:
       addl $10,num1
       mov num1,%rdi
       mov $60,%rax
       syscall
```

A few notes. `.long` reserves a 4-byte (32-bit) cell, hence `addl` (the `l` suffix means "long" = 32-bit). The size suffixes are `b` (byte = 1), `w` (word = 2), `l` (long = 4), `q` (quad = 8); usually the assembler can figure them out from operand sizes, but you have to be explicit when adding an immediate to a memory location whose size it can't guess.

Heads-up: `mov num1,%rdi` reads a 32-bit value from `num1` but the destination is a 64-bit register. This is exactly the "`.long` with 64-bit register" bug pattern I warn about every term. The cleaner fix is to either use `.quad` for 64-bit storage or `movl num1,%edi` for 32-bit, accepting the implicit zero-extension into `%rdi`. As written, you'll get the right answer on a fresh process because the upper bytes adjacent to `num1` happen to be zero, but it's a latent bug waiting to bite.

`data1.s` switches to RIP-relative addressing using `lea` (load effective address):

```
_start:
       ## rip-relative addressing
       ## lea is used for the & operation
       lea num1(%rip),%rbx # &num1 -> %rbx
       addl $10,(%rbx) # *(%rbx) = 10 + *(%rbx)
       mov num1,%rdi
       mov $60,%rax
       syscall
```

`lea num1(%rip),%rbx` is the assembly equivalent of `int* p = &num1;` — it puts *the address of* `num1` into `%rbx`. Then `(%rbx)` dereferences that pointer, just like `*p` in C. RIP-relative addressing is what you want in modern code because it's position-independent — the offset between the instruction and the data is fixed at link time.

`data2.s` extends the same idea to a four-element array and walks it with a loop:

```
       .section .data
arr1: .long 1,2,3,4

       .section .text
       .global _start

_start:
       lea arr1(%rip),%rbx # base of the array
       mov $0,%rcx # int i = 0
       mov $0,%rax # int sum = 0

loopBody:
       cmp $4,%rcx # i < 4
       jge done
       add (%rbx,%rcx,4),%rax
       ## *(%rbx + 4 * %rcx)
       ## arr1[rcx]
       add $1,%rcx # i++
       jmp loopBody

done:
       mov %rax,%rdi
       mov $60,%rax
       syscall
```

The juicy line is `add (%rbx,%rcx,4),%rax`: this is the "base + index · scale" addressing mode, with scale 4 because each element of `arr1` is 4 bytes (`.long`). It literally compiles `arr1[i]` into one instruction.

But here's another classic bug: `add (%rbx,%rcx,4),%rax` loads a 4-byte value into a 64-bit register (`%rax`), and `add` with no explicit suffix has to guess sizes. The strictly correct version uses `addl ...,%eax` for 32-bit math, or declares the array as `.quad` and uses scale 8. The example runs "correctly" because all our values are small, but the moment you're summing 32-bit values and care about the upper half of the accumulator, this bites. Same family as the `data0.s` caveat; it's the trade-off the file makes for the sake of a small example.

`cmp` and the conditional jumps

`cmp S, D` (almost) computes `D - S` and sets flags without storing the result anywhere. The conditional jumps then read those flags. Because the destination is on the right and the meaning of "less" is "$D < S$," the surface order in your head can be backwards from what you read off the page; this is the second source of all-term confusion (the first being the AT&T destination-on-right order).

`cmp1.s` is the warning shot:

```
_start:
       mov $0,%rdi
       mov $50,%r8
       mov $2,%r9
       cmp %r8,%r9 # really a subtract under the hood
       jl less
less:
       mov $10,%rdi #whoops! this line will always happen :(
       ## the above is if (r8 < r9) {rdi=10}

       mov $60,%rax
       syscall
```

The bug is structural: regardless of whether `jl` fires, control falls into `less:` on the next line. Labels are not function bodies; they're just named addresses. Without a `jmp end` between the conditional and the `less:` label, you always run the body.

`cmp2.s` adds the missing `jmp`:

```
_start:
       mov $0,%rdi
       mov $0,%r8
       mov $50,%r9
       cmp %r8,%r9
       jl less
       jmp end
less:
       mov $10,%rdi
end:
       mov $60,%rax
       syscall
```

Now the structure is genuinely an if-statement: "if $\%r9 < \%r8$, go set `%rdi = 10`; otherwise skip it." Since $50 \not< 0$, we don't take the branch and exit with status 0.

`cmp3.s` swaps in `jne` (jump if not equal) with `0 == 0`:

```
_start:
       mov $0,%rdi
       mov $0,%r8
       mov $0,%r9
       cmp %r8,%r9
       jne notequal
       jmp end
notequal:
       mov $10,%rdi
end:
       mov $60,%rax
       syscall
```

Predictably exits 0 (they're equal, so the branch isn't taken).

`cmp4.s` compares against `-1` instead and exits 10 (0 is not equal to -1).

The conditional jumps you'll meet most:

- `jmp` — unconditional.
- `je`, `jne` — equal, not equal.
- `jl`, `jle`, `jg`, `jge` — less, less-or-equal, greater, greater-or-equal (signed).

A loop in assembly

`loop1.s` sums the integers from 1 to 10 by counting down:

```
loopstart:
       cmp $0,%rcx
       jle end
       add %rcx,%rax # rax += rcx
       sub $1,%rcx # rcx--
       jmp loopstart

_start:
       mov $0,%rax
       mov $10,%rcx # the sum should be 55
       jmp loopstart

end:
       mov %rax,%rdi
       mov $60,%rax
       syscall
```

Two things to note. First, the loop label sits *above* `_start` in the file; that's fine, label position doesn't mean anything to control flow, only the `jmp loopstart` at the bottom of `_start` does. Second, this is the same shape as any "while" loop in C: a comparison at the top, a body, a step, and an unconditional jump back. Every loop you'll write in assembly follows this template.

55 in `%rdi`, exits with status 55.

4 Assembly — Linking, Recursion, and a Real Stack Frame

This section corresponds to the deeper assembly material in the `assembly/` directory. The goal across all of these files is to move from "some labels and a `_start`" to genuine multi-file, multi-function assembly programs that read input, do non-trivial work, and print results.

Using external functions

Up until now, every assembly program has been one self-contained `.s` file. Real programs are many files linked together, and once we have `call`/`ret` and the calling convention, linking is just declaration plumbing. The two helpers we'll lean on forever are `readInt` and `writeInt`.

`readInt.s`: reading an integer from stdin

```
    .section .bss
buffer: .zero 128

    .section .text
    .global readInt

parseInt:
intRead:
    movb (%rdi,%rcx,1), %r8b # Load one byte from buffer[%rcx] into %r8b
    sub $'0', %r8 # Convert ASCII digit to numeric value
    imul $10, %rax # multiply accumulator by 10
    add %r8, %rax # add this digit to accumulator
    inc %rcx
    cmp %rsi, %rcx
    jl intRead
    ret

readInt:
    mov $0, %rax # sys_read
    mov $0, %rdi # stdin
    lea buffer(%rip), %rsi
    mov $128, %rdx
    syscall

    mov %rax, %rbx # bytes read
    mov $0, %rcx
    mov $0, %rax
    lea buffer(%rip), %rdi
    dec %rbx # trim trailing newline
    mov %rbx, %rsi

    call parseInt
    ret
```

A few details worth pausing on:

- `.bss` is the section for uninitialized, zero-filled storage. It costs nothing on disk because the loader knows it just hands you zeroed pages.
- `movb (%rdi,%rcx,1), %r8b` loads a single byte of the input buffer into the *low byte* of `%r8`. There is a subtle bug pattern here: `movb` into `%r8b` leaves the upper bytes of `%r8` untouched, so any garbage sitting in `%r8` from before bleeds into your arithmetic. The robust idiom is `movzbl (%rdi,%rcx,1), %r8d` ("move byte to long, zero-extended"), which clears the upper bits for you. As written, `readInt` works because `%r8` happens to be zero on entry (the kernel zeros registers before `_start`), but the moment this function is called from somewhere that has used `%r8`, you'd see silent corruption.
- `readInt` also clobbers `%rbx` without saving it. `%rbx` is callee-saved in the Linux ABI, so a strictly correct `readInt` would push/pop it around the body. Again, the example works because `_start` doesn't expect anything in `%rbx`.

These are the kind of "works in this context, fragile elsewhere" shortcuts that you'll start spotting once you've written a few functions yourself.

`writeInt.s`: printing an integer to stdout

`writeInt` is the mirror image: take an integer in `%rdi`, convert it to a decimal string, write it to stdout. It's longer than `readInt` because it has to handle three slightly tricky cases: zero, negatives, and `INT_MIN`.

The structure is:

1. Standard prologue (`push %rbp; mov %rsp,%rbp; sub $40,%rsp`) carving out a 32-byte buffer on the stack plus a little extra.
2. If the input is zero, write a single `'0'` into the buffer and skip the conversion loop.
3. If the input is negative, set a flag and negate. The `neg` instruction sets the overflow flag if you try to negate -2^{63} (because $+2^{63}$ doesn't fit in 64-bit signed), so we check `jno` and special-case that with `movabsq $9223372036854775808, %rax` — a 64-bit immediate move, since the constant doesn't fit in a 32-bit sign-extended immediate.
4. Convert digit-by-digit: `div %rbx` with $\%rbx = 10$ gives quotient in `%rax` and remainder in `%rdx`; add `'0'` to the remainder, store it at the next free slot from the right end of the buffer, repeat until the quotient is zero.
5. If the negative flag is set, prepend a `'-'`.
6. `write` syscall: fd=1, buffer pointer in `%rsi`, digit count in `%rdx`.
7. Epilogue: `leave; ret`, where `leave` is shorthand for `mov %rbp,%rsp; pop %rbp`.

The writing-from-the-end trick is the right answer to "how do I print digits in the correct order when I generate them in reverse?" You leave a pointer at the end of the buffer, write each digit and back the pointer up by one; at the end, the pointer is sitting at the first character of your number.

`echoInt.s`: putting them together

`echoInt.s` is just "read an int, echo it back, with some prompts in between." Three message strings, three calls to the `write` syscall, plus calls to `readInt` and `writeInt`:

```
        .section .rodata
msg1: .ascii "Enter a number: "
msg1len = . - msg1
msg2: .ascii "Here's what you entered: "
msg2len = . - msg2
msg3: .ascii "\nGoodbye!\n"
msg3len = . - msg3

        .section .text
        .global _start
        .extern readInt
        .extern writeInt

_start:
        mov $1,%rax
        mov $0,%rdi
        lea msg1(%rip),%rsi
        mov $msg1len,%rdx
        syscall

        call readInt
        push %rax # whoops better save rax quick

        mov $1,%rax
        mov $0,%rdi
        lea msg2(%rip),%rsi
        mov $msg2len,%rdx
        syscall

        pop %rax # we got it back!!!
        mov %rax,%rdi
        call writeInt

        mov $1,%rax
        mov $0,%rdi
        lea msg3(%rip),%rsi
        mov $msg3len,%rdx
        syscall

        mov $60,%rax
        xor %rdi,%rdi
        syscall
```

Heads-up: `mov $0,%rdi` just before each `write` syscall is wrong. `write`'s first argument is the file descriptor; `0` is stdin, not stdout. We meant `mov $1,%rdi` (fd 1 = stdout). On Linux you may still see the message because writing to stdin sometimes "works" when stdin is a terminal (the terminal is bidirectional), but it's wrong and will fail or misbehave the moment you redirect input. This is a recurring bug pattern — when you copy the `write` syscall shape, double-check that `%rdi` is 1 and not 0.

The other interesting move is `push %rax` / `pop %rax` around the second `write`. Because `write` clobbers `%rax` (with its return value, the byte count), we shove the `readInt` result onto the stack

to keep it safe.

`echoInt2.s`: refactor into a helper

`echoInt2.s` is the same program with all the `write` boilerplate factored into a tiny `printStr` function:

```
printStr:
       mov %rsi,%rdx
       mov %rdi,%rsi # load message address
       mov $1,%rax
       mov $0,%rdi
       syscall
       ret
```

Convention used here: address of string in `%rdi`, length in `%rsi`. The body shuffles those into `%rsi` and `%rdx` where `write` wants them, then issues the syscall.

The same fd-0-vs-fd-1 bug is present in `printStr` (`mov $0,%rdi` should be `mov $1,%rdi`); call it out the first time you see it in your own code.

The `_start` body is now beautifully clean:

```
_start:
       lea msg1(%rip),%rdi
       mov $msg1len,%rsi
       call printStr

       call readInt
       push %rax

       lea msg2(%rip),%rdi
       mov $msg2len,%rsi
       call printStr

       pop %rax
       mov %rax,%rdi
       call writeInt

       lea msg3(%rip),%rdi
       mov $msg3len,%rsi
       call printStr

       mov $60,%rax
       xor %rdi,%rdi
       syscall
```

This is *the* reason functions exist, and it shows up just as hard in assembly as it does in C: you stop repeating yourself, and each piece of the program does one thing.

Recursion: `fibber.s`

`fibber.s` is our first recursive function in assembly. $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ with $\text{fib}(0) = 0$, $\text{fib}(1) = 1$.

```
fib:
    push %rbp
    mov %rsp, %rbp
    sub $16, %rsp # 16 bytes for one local + alignment

    cmp $1, %rdi
    jle .base

    push %rdi # save n
    dec %rdi # n-1
    call fib # %rax = fib(n-1)
    mov %rax, -8(%rbp) # stash fib(n-1) in our local
    pop %rdi # restore n

    push %rdi
    dec %rdi
    dec %rdi # n-2
    call fib # %rax = fib(n-2)
    pop %rdi

    add -8(%rbp), %rax # %rax = fib(n-1) + fib(n-2)

    mov %rbp, %rsp
    pop %rbp
    ret

.base:
    mov %rdi, %rax
    mov %rbp, %rsp
    pop %rbp
    ret
```

Two patterns to internalize. First, `push %rdi` before each recursive call: `%rdi` is caller-saved, so the recursive call is allowed to clobber it. We need `n` again afterward, so we park it on the stack. Second, `mov %rax, -8(%rbp)` stashes the first sub-result in a local stack slot before we make the second recursive call, because that second call will overwrite `%rax` with its return value. Each recursive activation has its own `%rbp`, so each one has its own `-8(%rbp)` slot — that's how recursion "just works" once you're using the stack properly.

The `sub $16,%rsp` carves out 16 bytes when we only need 8; the extra is to keep `%rsp` 16-byte aligned at the next `call`. The ABI requires that alignment, and SSE instructions in libraries can fault if it's wrong.

Using `leave`: `fibber2.s`

`fibber2.s` is identical to `fibber.s` except for two small tweaks worth noting (the rest is verbatim).

First, in the second recursive branch, the `push %rdi` / `pop %rdi` around the inner `call fib` is gone:

```
    # Compute fib(n - 2)
    dec %rdi # now n - 1
    dec %rdi # now n - 2
```

```
    call fib # %rax = fib(n-2)

    # Add fib(n - 1) and fib(n - 2)
    add -8(%rbp), %rax
```

That's a deliberate "you don't need to save what you don't need." After the second recursive call we don't use `%rdi` again, so there's no point in saving it.

Second, you could replace the explicit `mov %rbp,%rsp; pop %rbp` epilogue with the single `leave` instruction, which does exactly that. (The file as written still has the long form; `leave` is what most modern compilers emit and what we'll use in `stacky.s`.)

The big takeaway: you don't have to pay for callee-save dance you don't need. Save things only when you need to read them after the call.

Stack frames with locals: `stacky.s`

`stacky.s` is the most "real-feeling" assembly we'll write this term: a function that takes two arguments, declares two locals on the stack, computes with them, and returns. The C version it's mimicking:

```c
long fun(long a, long b){
  long c = a + b;
  long d = 2 + b;
  return (c + d);
}
```

The assembly:

```
fun:
        # Standard function prologue
        push %rbp
        mov %rsp,%rbp
        # Allocate 16 bytes of local stack space
        sub $16,%rsp

        # First local: c = a + b at -8(%rbp)
        mov %rdi,-8(%rbp)
        add %rsi,-8(%rbp)

        # Second local: d = b + 2 at -16(%rbp)
        mov %rsi,-16(%rbp)
        addq $2,-16(%rbp)

        # Return c + d
        mov -16(%rbp),%rax
        add -8(%rbp),%rax

        # 'leave' = mov %rbp,%rsp; pop %rbp
        leave
        ret
```

Walk through this on paper once. Sketch the stack growing *downward* from the top of memory; mark where `%rbp` lands after the prologue; mark `-8(%rbp)` and `-16(%rbp)` as your two locals; then trace each instruction. Once you've done it once, every other stack-frame function will look the same.

The `_start` that drives it reads two ints, calls `fun` with them as arguments, prints the result, and exits. The bit worth flagging is the argument plumbing: `fun`'s arguments are in `%rdi` and `%rsi` per the calling convention, but `readInt` returns its result in `%rax`, so we have to shuffle:

```
        call readInt
        push %rax # save first input

        ...

        call readInt # second input now in %rax

        # Set up arguments for fun()
        mov %rax,%rsi # second arg
        pop %rax # restore first input
        mov %rax,%rdi # first arg

        call fun
```

That `push`/`pop` around the second `readInt` is the same trick as in `echoInt.s`: any time you have a value you care about and you're about to call a function that clobbers the register it lives in, the stack is the right scratch pad.

For deeper coverage of all of this, the canonical reference is `assemblyGuide.org` in the repo.

5 Forking — Processes, `fork()`, `wait()`, and Exit Status

A process is a context of execution that the operating system controls and keeps track of. In ye olden days, only one process at a time could run; the kernel takes turns by *context switching*, saving and restoring all of a process's state (registers, memory, stack, file descriptors, etc.). Each process gets to pretend it lives in its own little world, and processes are *isolated* — they don't share memory or registers or state.

`fork()` duplicates the current process. Both the parent and the child continue from the same line, but the child gets its own copy of memory and runs independently. This was the original way Unix gave you concurrency, and it's still in active use today (every shell pipeline, every web server worker, etc.).

The code for the examples in this section actually lives in `spring2025/threading/` alongside the pthread code (the `forking/` directory in the repo is just an agenda); it's the right code regardless.

The simplest possible fork: `fork1.c`

```c
int main(){
  // only one process running here
  fork();
  // now there are two
```

```
  printf("This message should be printed by parent and child\n");

  return 0;
}
```

Run it. You'll see the message twice. There's only one `printf` in the source, but after `fork()` returns, there are two processes both sitting on the next line, both running that `printf`.

Telling parent and child apart: `fork2.c`

`fork()` returns different things in the parent and child. Specifically:

- In the parent: the PID of the newly created child (a positive integer).
- In the child: 0.
- On error: -1.

`fork2.c` uses that to branch:

```
int main(){
  pid_t pid = fork();

  if(pid != 0){
    printf("I'm the parent!\n");
  }
  else{
    printf("I'm the child!\n");
  }
  printf("This message should be printed by parent and child\n");

  return 0;
}
```

The pattern of writing one program that bifurcates based on `pid` is the canonical `fork` idiom.

Synchronization with `wait()`: `fork3.c`

When a child finishes, the parent has to acknowledge it (`wait()`) or it sticks around as a *zombie process* — its exit status is sitting in the kernel waiting to be reaped, and the PID isn't reusable. `fork3.c`:

```
int main(){
  pid_t pid = fork();

  if(pid != 0){
    // anti-zombie ward
    wait(NULL);
    printf("I'm the parent!\n");
  }
  else{
    printf("I'm the child!\n");
  }
  printf("This message should be printed by parent and child\n");
```

```
  return 0;
}
```

`wait(NULL)` blocks the parent until the child terminates, and because we're not interested in the exit status here we pass `NULL`. The order of output is now deterministic: the child runs to completion before the parent's branch resumes. (This is the "missing `wait()`" fix — forgetting it is one of the most common bugs in fork-based programs.)

Reading the exit status: `fork4.c`

If you do care about the child's exit code, pass `wait()` a pointer to an `int` and use the `WEXITSTATUS` macro to extract the actual status:

```
int main(){
  pid_t pid = fork();

  if(pid != 0){
    int result;
    printf("I'm the parent!\n");
    wait(&result);
    // wexitstatus(r) ==> (r >> 8) & 255
    printf("My child returned: %d\n",WEXITSTATUS(result));
  }
  else{
    printf("I'm the child!\n");
    return 2;
  }

  return 0;
}
```

The reason `WEXITSTATUS` exists is that the integer `wait` fills in is encoded: the low byte holds signal/termination info, and the actual exit status lives in the next byte up. Hence `(r >> 8) & 255`. If you read `result` directly without the macro, you'd get a number that looks like the exit status times 256 — a very specific, very memorable bug.

Routing exit status through user input: `fork5.c`

`fork5.c` pushes the same idea a step further: the child reads input and sets its exit status from whether the input parsed:

```
int main(){
  pid_t pid = fork();
  int returned;

  if(pid == 0){
    // in the child process
    int blah;
    int results;
    printf("Say somethin', will ya: ");
    results = scanf("%d",&blah);
    if(results < 1){
```

```
      return 1;
    }
    else {
      return 0;
    }
  }
  else {
    // in the parent process
    wait(&returned);
  }
  printf("This was returned: %d\n",WEXITSTATUS(returned) );

  return 0;
}
```

Type a number and the parent reports 0; type letters and it reports 1. The parent and child genuinely run concurrently — the parent sits in `wait()` while the child sits in `scanf()`.

Memory really is copied: `forkVar.c`

`forkVar.c` is the demonstration that the parent and child have their own copies of every variable:

```
int main(){
  int thingy = 100;
  pid_t pid = fork();
  if(pid != 0){
    thingy++;
    thingy++;
    printf("Thingy: %d\n",thingy);
  }
  else{
    thingy++;
    printf("Thingy: %d\n",thingy);
  }
  return 0;
}
```

The parent prints 102, the child prints 101. They share *nothing* mutable. (Under the hood, Linux is doing copy-on-write for efficiency, but semantically they each have their own `thingy`.) This is the headline difference between fork and threads, which we'll get to next: with threads, both "branches" would be modifying the same `thingy`, and the answer would depend on scheduling.

For more fork patterns and IPC mechanisms (pipes, named pipes, shared memory), see `concurrencyGuide.org` in the repo.

6 Threading — pthreads, Race Conditions, and Mutexes

The big mental jump from forking to threading: a thread is part of the same process, so multiple threads share *the same memory*. That's both the appeal (cheap, fast, easy data sharing) and the danger (concurrent access to shared variables is a minefield).

Forks vs. threads

- A *process* has its own memory, its own registers, its own state. Processes are isolated.
- A *fork* creates a whole new process; parent and child have separate copies of everything.
- A *thread* is a strand of execution *within* a process. Threads have their own registers and their own stack, but they share the heap, globals, and file descriptors with every other thread in the same process.

So if I fork and increment a global, only the child sees the change. If I spin up a thread and increment a global, the main thread sees the change too — and so does every other thread that happens to be reading it at the same time, which is where the trouble starts.

Creating threads: `thread1.c`

```c
#include <pthread.h>

void* ourPrinter(void* arg){
  char* msg = (char*) arg;
  printf("Our thread says: %s", msg);
  return NULL;
}

int main(){
  pthread_t thread1;
  pthread_t thread2;

  char* msg1 = "Hi there I'm one thread\n";
  char* msg2 = "Hi there I'm a different thread\n";

  pthread_create(&thread1,NULL,ourPrinter,msg1);
  pthread_create(&thread2,NULL,ourPrinter,msg2);

  pthread_join(thread1,NULL);
  pthread_join(thread2,NULL);

  return 0;
}
```

A thread function always has the type `void* f(void*)` — `void*` is the wildcard pointer in C, so this lets you pass *anything* in and get *anything* out, at the cost of having to cast at both ends.

`pthread_create` takes:

1. A pointer to a `pthread_t` that will be filled in with the new thread's identity,
2. Attributes (`NULL` for defaults),
3. The function the thread will run,
4. A `void*` argument that gets passed to that function.

`pthread_join` blocks the calling thread until the named thread finishes, like `wait()` for processes. Compile with `-pthread`: `gcc -pthread thread1.c -o thread1`.

Returning a value: `threadResult.c`

The fourth-argument-in is mirrored by a return-via-second-arg-of-join on the way out:

```
void* weirdFunction(void* arg){
  (void)arg;
  int* sleepPointer = malloc(sizeof(int));
  *sleepPointer = rand()%5+1;
  sleep(*sleepPointer);
  return sleepPointer;
}

int main(){
  srand(time(0));
  pthread_t thread1;
  pthread_t thread2;
  void* res1;
  void* res2;

  pthread_create(&thread1,NULL,weirdFunction,NULL);
  pthread_create(&thread2,NULL,weirdFunction,NULL);

  pthread_join(thread1,&res1);
  pthread_join(thread2,&res2);

  printf("Thread 1 did a sleep for %d seconds\n",*(int*)res1);
  printf("Thread 2 did a sleep for %d seconds\n",*(int*)res2);
  free((int*)res1);
  free((int*)res2);
  return 0;
}
```

Subtle but important: the function returns a pointer to heap-allocated memory rather than to a local variable. If you returned the address of a stack local, the storage would be gone the moment the function returned. `malloc`-ed memory persists until you `free` it, which is why this pattern is safe.

Race conditions: `threadRace1.c`

The dark side of shared memory:

```
int ourCounter = 0;

void* threadCounter(void* arg){
  (void)arg;
  // retrieve the value here
  int temp = ourCounter;

  sleep(rand()%3+1);
  // store the incremented value down here
  ourCounter = temp +1;
  return NULL;
}
```

```c
int main(){
  srand(time(0));
  pthread_t threads[10];

  for(int i = 0; i < 10; i++){
    pthread_create(&threads[i],NULL,threadCounter,NULL);
  }
  for(int i = 0; i < 10; i++){
    pthread_join(threads[i],NULL);
  }

  printf("What's the value of the counter? %d\n",ourCounter);

  return 0;
}
```

Naive expectation: ten threads each incrementing the counter, final value 10. Actual answer: usually 1.

The bug is the gap between the load (`int temp = ourCounter;`) and the store (`ourCounter = temp + 1;`). The `sleep()` in the middle is just there to make the race obvious — in real programs, the gap is shorter but the same race exists. While our thread is sleeping with `temp = 0`, every other thread is also reading 0 and waking up to write 1. Each thread independently writes 1, so the final value is 1 no matter how many threads ran.

This is the canonical *race condition*: multiple threads reading and writing the same memory without coordination.

Fixing it with a mutex: `threadmutex.c`

A *mutex* (mutual exclusion lock) lets exactly one thread at a time hold the lock; any other thread that tries to acquire it blocks until it's released. Wrap the critical section — the chunk of code that touches the shared variable — in lock/unlock and the race disappears:

```c
int ourCounter = 0;
pthread_mutex_t mutex;

void* threadCounter(void* arg){
  (void)arg;
  pthread_mutex_lock(&mutex);
  int temp = ourCounter;
  sleep(rand()%3+1);
  ourCounter = temp+1;
  pthread_mutex_unlock(&mutex);
  return NULL;
}

int main(){
  srand(time(NULL));
  pthread_t threads[10];
  pthread_mutex_init(&mutex,NULL);

  for(int i=0; i < 10; i++){
```

```
    pthread_create(&threads[i],NULL,threadCounter,NULL);
  }
  for(int i=0; i < 10; i++){
    pthread_join(threads[i],NULL);
  }
  pthread_mutex_destroy(&mutex);

  printf("The value of ourCounter is: %d\n",ourCounter);

  return 0;
}
```

Now the result is deterministically 10. Mutexes are the simplest synchronization primitive worth knowing; counting semaphores, condition variables, and read-write locks all build on the same "serialize access to shared state" idea but with more flexibility.

The lifecycle is: `pthread_mutex_init` before any thread might use it, then `lock`/`unlock` pairs around every critical section, then `destroy` after every thread that might use it has finished.

Designing the assignment

Assignment 3 (the threading assignment) asks you to:

- Set as constants the number of threads, the chunk size, and the array size (something massive — a million or ten million elements).
- Each thread, while there's still work to be done, claims a chunk of the array, computes on that chunk, then tries to grab another. When no chunks are left, the thread exits.
- The computation itself doesn't matter. Increment every element by one. Invert the colors of an image. Pick something.

Hint: you need *a* mutex — one mutex protecting the "where's the next chunk?" counter. The chunks themselves don't need locking because each thread only touches its claimed chunk.

For more sophisticated synchronization (semaphores, condition variables, dining philosophers), see the threading and synchronization sections of `concurrencyGuide.org`.

7 In-Class — April 10, 2025: Shifts, Registers, and `malloc`

We had one synchronous in-class meeting this term, on April 10. The agenda was short and the conversation was driven by questions, so this section is correspondingly compact.

Why is bit-shift multiplication or division?

A small worked example with 8-bit numbers:

$$001101_2 = 13$$

Shifting left by one:

$$001101 \ll 1 = 011010_2 = 2 + 8 + 16 = 26 = 2 \cdot 13.$$

That's the pattern: $n \ll k$ moves every bit k places higher, which multiplies its weight by 2^k. For unsigned values it's exact multiplication by 2^k.

A reminder of how to negate in two's complement:

1. Invert all the bits.
2. Add 1.

So $13 = 001101$ becomes 110010 after inversion and 110011 after adding 1; that's the two's complement encoding of -13.

This came back up in `shiftnegative.c`, which exercises the same `printBits`/`flipBit` helpers from `bitMunger.c` but with an unsigned right shift on -1:

```
int main(){
  unsigned int num = -1; // 1111111..11111
  printf("Our number before shifting is: %d\n",num);
  printBits(num);
  num = num >> 3;
  printf("Our number after shifting is: %d\n",num);
  printBits(num);
  return 0;
}
```

`-1` in 32-bit two's complement is all ones. Shifting right by 3 in an *unsigned* variable does a *logical* shift (zeros in from the left), so the result is $2^{29} - 1 + \ldots$ (the top three bits become zero, the rest stay ones). The print as `%d` reinterprets that as signed, so you see a large positive number. The same right shift on a *signed* `int` would do an arithmetic shift (sign-bits in from the left) and the value would stay -1. Logical vs. arithmetic shift only matters for negatives, and signedness of the variable picks which one C uses.

Registers, more carefully

A few questions came up about how to think of registers.

- There are 16 of them on each core, and each holds 64 bits = 8 bytes. So one core has $16 \cdot 8 = 128$ bytes of register storage.
- They can be accessed in a single clock cycle. A 1 GHz processor has a clock cycle of one billionth of a second ($1\,\text{Hz} = 1/\text{s}$, so $1\,\text{GHz} = 10^9/\text{s}$).
- They aren't really like variables. They're the scratch paper of the processor. Almost every instruction operates on registers, so values flow in and out of them constantly.
- A core is the physical thing that runs code. Modern processors have many cores, each with its own register file.
- Memory is roughly 10 000 to 100 000 times slower than register access; caching narrows that gap considerably for hot data, but the cost ordering is still register $\to$ L1 $\to$ L2 $\to$ L3 $\to$ main memory $\to$ disk, with each step roughly an order of magnitude slower than the one before.

How many variables do you need to "saturate" the registers? Naively 16, but in practice anything you want to keep across a function call or a loop iteration has to live in a register at some point, so register pressure is one of the main things compilers worry about during optimization.

Two quick `malloc` examples

I owed everyone a couple more concrete `malloc` examples after video 1, so we walked through two.

`malloc1.c` is one struct on the heap:

```
struct Garbage {
  int n1;
  int n2;
};

int main(){
  struct Garbage* g =
    (struct Garbage*) malloc(sizeof(struct Garbage));

  g->n1 = 10;
  g->n2 = 20;

  printf("g->n1 is %d\n",g->n1);
  printf("g->n2 is %d\n",g->n2);

  return 0;
}
```

`malloc` takes a size in bytes and gives back a pointer to a region of that size. Unlike `new` in C++, it doesn't know anything about your type, so you pass `sizeof(struct Garbage)` and cast the resulting `void*`. The `->` arrow is just shorthand for "dereference, then access a field" (`g->n1` is `(*g).n1`).

`malloc2.c` is an array of structs:

```
int main(){
  struct Garbage* g =
    (struct Garbage*) malloc(10 * sizeof(struct Garbage));

  g[0].n1 = 10;
  g[0].n2 = 20;

  printf("g[0].n1 is %d\n",g[0].n1);
  printf("g[0].n2 is %d\n",g[0].n2);

  free(g);
  return 0;
}
```

Once you have a pointer from `malloc`, you index it like an array. `g[0]` is the first `struct Garbage`; `g[0].n1` reaches into the `n1` field of that struct. And `free(g)` releases the whole 10-element block in one go — you don't free element-by-element, because `malloc` allocated the whole block as a single allocation.

That's where this term ends. The asynchronous format means you're free to revisit any of these in any order; the videos and the agenda files in each subdirectory are the canonical pairing for each chunk.